

コマ 8: 関数呼び出し・再帰

今日のゴール

ユーザー定義関数の定義・呼び出し・再帰呼び出しを実装する。

第 07 回までは、`main` 関数だけを扱ってきた。この回では、複数の関数を定義し、それらを読み出せるようにする。

```
int fib(int n) {
    if (n <= 1) {
        return n;
    }
    return fib(n - 1) + fib(n - 2);
}

int main() {
    return fib(10);
}
```

目標は、引数の受け渡し、関数呼び出し `call`、戻り値の受け取り、再帰呼び出しを正しく生成できるようにすることである。

1 この回で扱う範囲

対象にするプログラムは、`main` に加えてユーザー定義関数を含むものに限定する。

種類	例
関数定義	<code>int add(int a, int b) { return a + b; }</code>
関数呼び出し	<code>add(3, 5)</code>
再帰呼び出し	<code>fib(n - 1)</code>
相互再帰	<code>is_even</code> / <code>is_odd</code> の互いの呼び出し
関数宣言	<code>int is_even(int n);</code>
最大 8 個までの引数	<code>clamp(val, lo, hi)</code>

これまでに扱ったすべての機能（変数、式、制御構文）を、`main` 以外の関数内でも使用できる。

2 AST を確認する

2.1 関数定義

```
python3 scaffold/parse_viewer.py sessions/08_functions_recursion/tests/target.c
```

このプログラムの内容は次の通り。

```
int fib(int n) {
    if (n <= 1) {
        return n;
    }
    return fib(n - 1) + fib(n - 2);
}

int main() {
    return fib(10);
}
```

```
(program
  (funcdef "fib" :type int
    (params (param "n" :type int))
    (block
      (if
        (cond
          (le (var "n") (num 1)))
        (then
          (block
            (return (var "n")))))
        (return
          (add
            (call "fib"
              (args
                (sub (var "n") (num 1))))
            (call "fib"
              (args
```

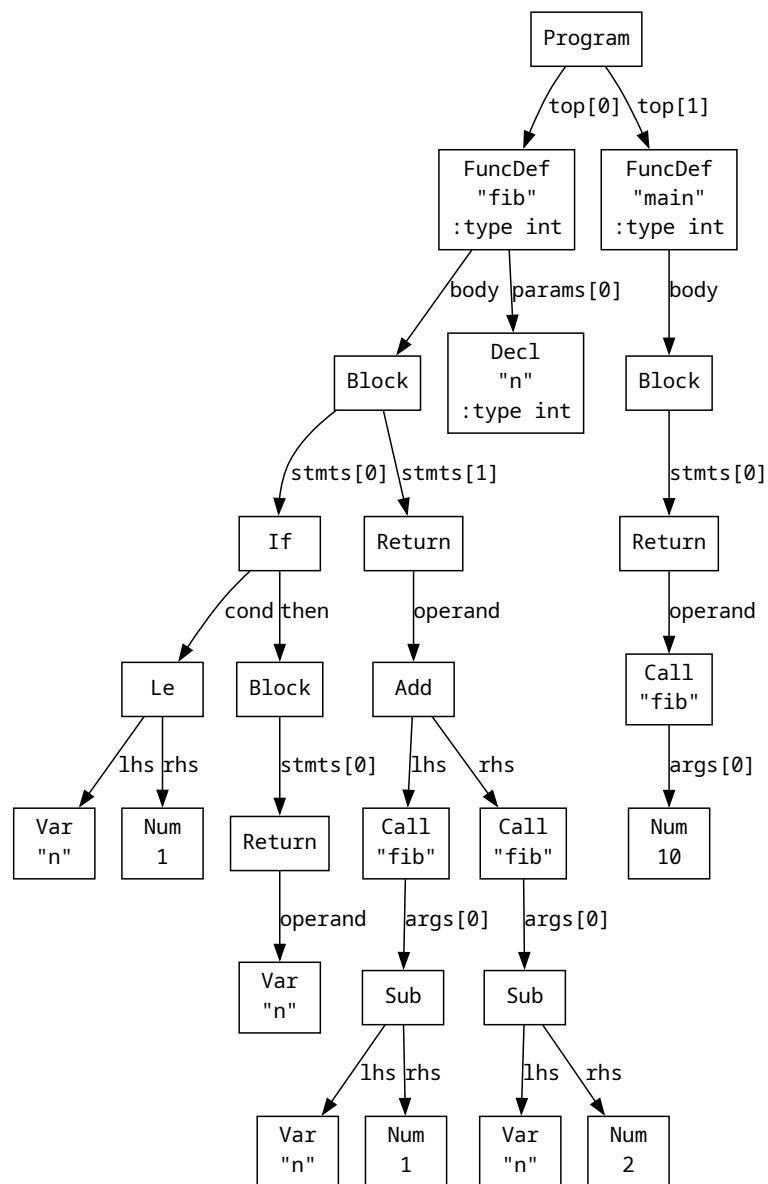


図 1 関数定義の AST

```

        (sub (var "n") (num 2))))))
(funcdef "main" :type int (params)
  (block
    (return
      (call "fib"
        (args (num 10)))))))

```

新しく登場するノードは次の通り。

S 式	Node での表現	意味
(call "f" (arND_CALL))		関数 f の呼び出し
(param "n" :tND_FUNCDEF.params[0])		関数のパラメータ名と型
(funcproto "fND_FUNCPROTO...")		関数宣言

ND_CALL は式である。codegen() で処理し、結果はa0 に置く。

2.2 相互再帰の AST

```
python3 scaffold/parse_viewer.py sessions/08_functions_recursion/tests/mutual_rec.
```

このプログラムの内容は次の通り。

```
int is_even(int n);
int is_odd(int n);

int is_even(int n) {
    if (n == 0) {
        return 1;
    }
    return is_odd(n - 1);
}

int is_odd(int n) {
    if (n == 0) {
        return 0;
    }
    return is_even(n - 1);
}

int main() {
    return is_even(10) + is_odd(7) * 10;
}
```

```

(program
  (funcproto "is_even" :type int
    (params (param "n" :type int)))
  (funcproto "is_odd" :type int
    (params (param "n" :type int)))
  (funcdef "is_even" :type int
    (params (param "n" :type int))
    (block
      (if
        (cond
          (eq (var "n") (num 0)))
        (then
          (block
            (return (num 1))))))
      (return
        (call "is_odd"
          (args
            (sub (var "n") (num 1))))))))
  (funcdef "is_odd" :type int
    (params (param "n" :type int))
    (block
      (if
        (cond
          (eq (var "n") (num 0)))
        (then
          (block
            (return (num 0))))))
      (return
        (call "is_even"
          (args
            (sub (var "n") (num 1))))))))
  (funcdef "main" :type int (params)
    (block
      (return
        (add
          (call "is_even"
            (args (num 10)))
          (mul
            (call "is_odd"
              (args (num 7)))
            (num 10)))))))

```

相互に呼び合う関数では、先に `funcproto` が出る。これは、宣言より前に関数定義を書く必

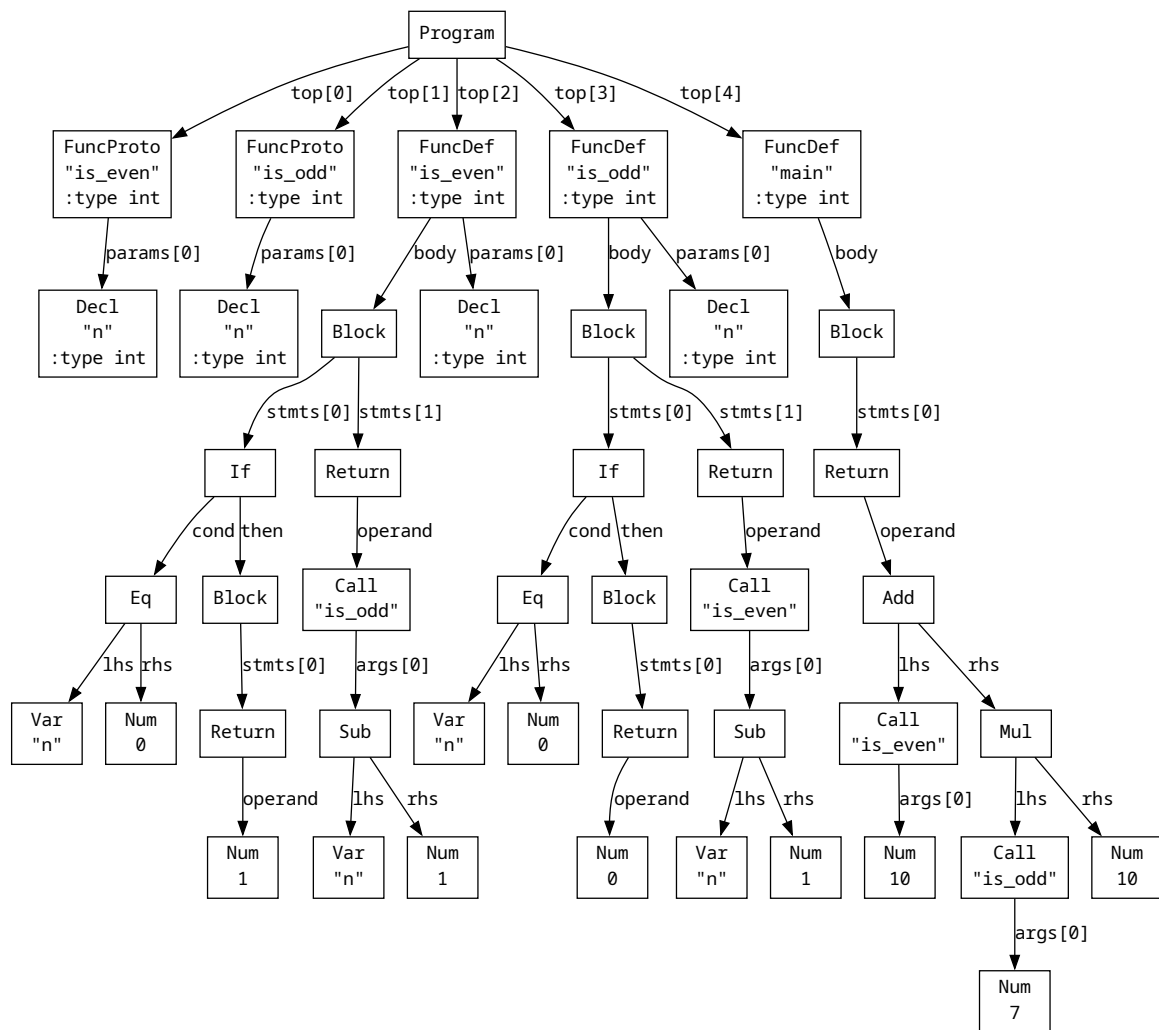


図 2 相互再帰の AST

要がある場合に、Parser が宣言を前方に移動するためである。

3 RV64 の関数呼び出し規約

関数呼び出しでは、呼び出す側と呼び出される側の間で、どのレジスタを何に使うかを決めておく必要がある。この取り決めを ABI の一部として扱う。

この回で使う主な約束は次の通り。

役割	レジスタ	説明
第 1 引数	a0	呼び出し側がセットする
第 2 引数	a1	同上
第 3 引数	a2	同上。以下 a7 まで
戻り値	a0	呼び出された側がセットする
戻り先アドレス	ra	call 命令が自動セットする
フレームポインタ	s0	呼び出し前後で同じ値に復元する
スタックポインタ	sp	call 時に 16 バイト境界に揃える

関数を呼び出すとき、`call f` という命令を使う。

```
call fib
```

この命令は、次の 2 つのことを行う。

1. 次の命令のアドレスを `ra` に保存する
2. `fib` ラベルへジャンプする

呼び出された関数は、最後に `ret` を実行する。`ret` は `ra` に保存されたアドレスへ戻る。

したがって、再帰を含む関数呼び出しを行うときは、自らの `ra` を関数の先頭で保存し、末尾で復元する必要がある。これは第 07 回のプロローグ・エピローグがすでに行っている。

関数を呼ぶたびに新しいフレームが低いアドレス側へ積まれ、`ret` するたびに 1 つ上のフレームへ戻る。各呼び出しが自分専用の引数スロットを持つため、再帰しても `n` の値が混ざらない。

4 引数を受け取る側

関数が呼ばれた直後、引数は `a0`, `a1`, … に入っている。

```
int add(int a, int b) {
    return a + b;
}
```

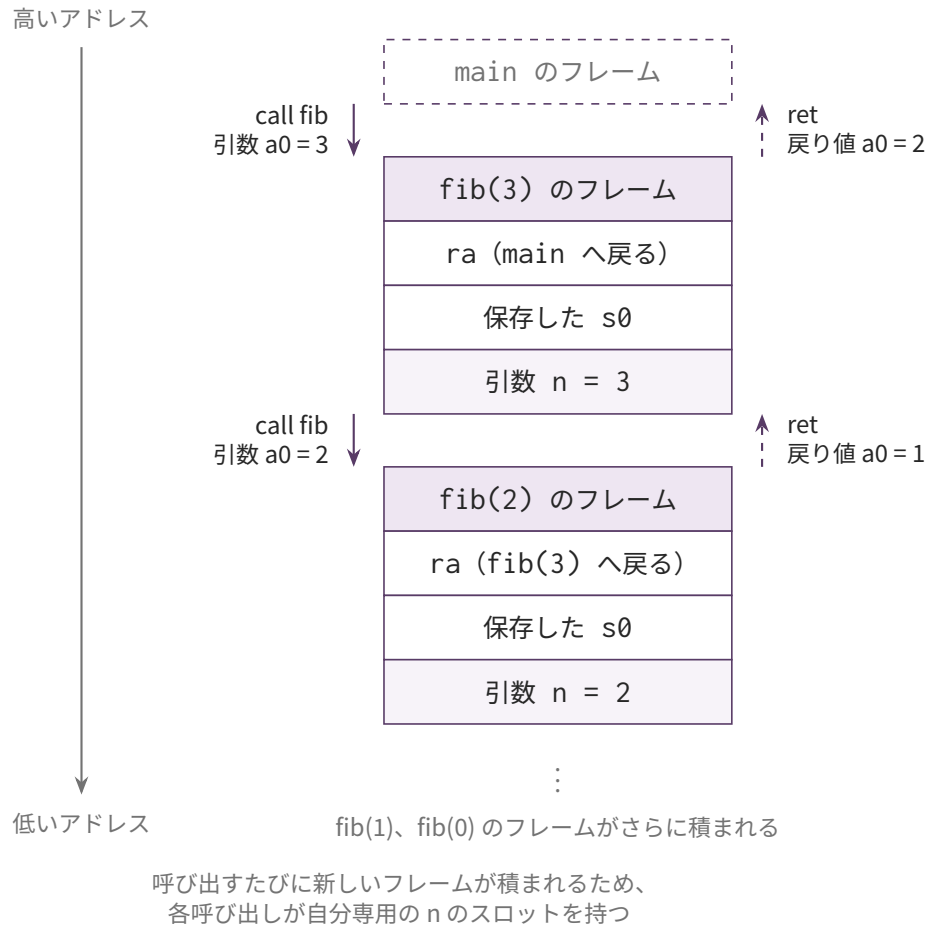


図 3 fib(3) を呼び出したときのスタックフレームの積み重なり

この関数が `add(3, 5)` と呼ばれた場合、関数に入った直後は次の状態になる。

レジスタ	内容
a0	第 1 引数 3
a1	第 2 引数 5

ただし、以後の計算で `a0` や `a1` はすぐ上書きされる。そのため、関数の先頭で引数をローカル変数と同じようにスタックへ保存する。

`gen_func()` では、ローカル変数を集める前に、まず `funcdef.params` を `self.alloc_local()` で登録する。

1. `params` を `self.alloc_local()` する
2. 関数本体内のローカル変数を `self.collect_decls()` する
3. `frame_size` を計算する
4. プロローグを出す
5. `a0`, `a1`, ... を対応する引数スロットへ `sd` する

たとえば `int add(int a, int b)` なら、次のように配置できる。

```
s0 - 24    param a
s0 - 32    param b
```

プロローグ後に、引数レジスタを保存する。

```
sd a0, -24(s0)
sd a1, -32(s0)
```

これ以降、引数 `a`, `b` は通常のローカル変数と同じように `codegen_lval()` でアドレスを取り、`ld` で値を読める。

5 関数を呼び出す側

関数呼び出し `f(x, y)` では、呼び出す直前に次の状態を作る。

レジスタ	内容
<code>a0</code>	第 1 引数
<code>a1</code>	第 2 引数
...	...

その後、`call f` を出す。

関数から戻ってくると、戻り値は `a0` に入っている。したがって、`codegen(ND_CALL)` の結果も他の式と同じく `a0` に残る。

6 codegen_Call の生成パターン

引数を左から順に評価すると、各引数の結果は毎回 `a0` に入る。そのままだと、次の引数を評価したときに前の引数が上書きされる。

そのため、各引数を評価したら一度スタックに保存する。

`codegen_Call(node)` は次の流れで実装する。

1. 引数を左から順に `self.codegen` する
2. 各引数の結果をスタックに積む (`_push_a0`)
3. スタックから `a0, a1, ...` にロードする (積んだ逆順)
4. 引数個数 * 8 でスタックを戻す
5. `sp` を 16 バイト境界へ揃える (次節。積んでいる一時値が奇数個なら 8 詰める)
6. `call 関数名` を出す
7. 5 で詰めた分を戻す

引数を `arg0, arg1, arg2` の順に `push` すると、最後に `push` したものが一番上に来る。

```
sp + 0   arg2
sp + 8   arg1
sp + 16  arg0
```

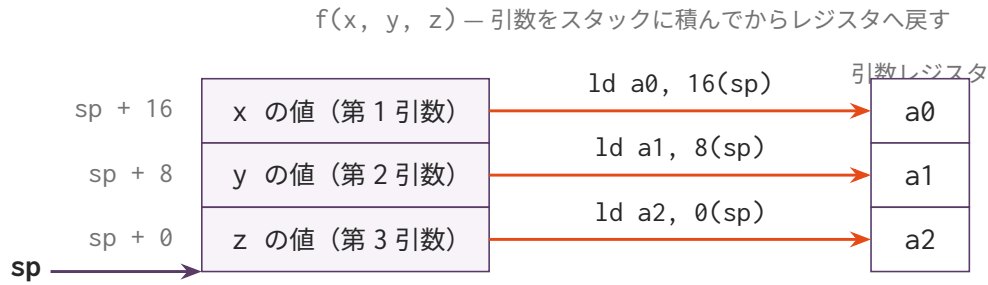
そのため、レジスタへ戻すときは次のように読む。

レジスタ	読む位置
<code>a0</code>	<code>sp + (N - 1) * 8</code>
<code>a1</code>	<code>sp + (N - 2) * 8</code>
...	...

最後に `push` した引数が `sp + 0` に来るため、レジスタへは逆順に読み出す。

7 call 前のスタックアラインメント

RV64 の呼び出し規約では、`call` する時点で `sp` が 16 バイト境界に揃っている必要がある。違反すると `Illegal instruction` や `Bus error` になり、原因の特定が難しい。



引数は左から順に評価して push するため、最後に push した z が sp + 0 に来る
 レジスタへ戻したら `addi sp, sp, 24` で引数の分を戻す
 そのあと、積んだままの一時値が奇数個なら 8 詰めてから `call f` を出す

図 4 3 引数の呼び出しで、push した引数をレジスタへ戻す対応

sp を動かしているのは次の 2 つだけである。

動かす場所	量
プロローグ	<code>frame_size + 16</code> <code>(align_to(_stack_offset, 16)</code> を通すので 16 の倍数)
<code>_push_a0() / _pop_into()</code>	1 回あたり 8 バイト

プロローグは常に 16 の倍数なので、`call` の時点で `sp` がずれるかどうかはそのとき何個の一時値を積んでいるかで決まる。

call 時の `sp` のずれ = 積んでいる一時値の個数 × 8
 → 奇数個なら 8 バイトずれている

ここで注意したいのは、引数の個数ではなく、呼び出しを囲む式が積んだ一時値が効くという点である。`codegen_Call()` は積んだ引数をレジスタへ戻して `sp` も戻すので、引数の push は `call` の時点では消えている。残るのは外側の式の分である。

`return n * fact(n - 1);` // * の左辺 `n` を 1 個積んだまま `fact` を呼ぶ → 8 ずれる

そこで、積んでいる個数を数えておき、`call` の直前で判定する。

```

_push_a0() で depth += 1
_pop_into() で depth -= 1

# codegen_Call の最後、引数を戻したあと
pad = 8 if depth is odd else 0
self.emit(f" addi sp, sp, -{pad}") ← 奇数のときだけ
self.emit(f" call {name}")
self.emit(f" addi sp, sp, {pad}") ← 呼び出しから戻ったら元へ

```

`depth` は関数ごとに 0 に戻す。関数を出し終えた時点で 0 でなければ `push` と `pop` の数が合っていないので、その場で気づける。

8 再帰呼び出しで壊れやすい点

次の式を考える。

```
return fib(n - 1) + fib(n - 2);
```

左側の `fib(n - 1)` を呼ぶと、戻り値は `a0` に入る。しかし、右側の `fib(n - 2)` を呼ぶと、その呼び出しでも `a0` が使われる。つまり、左側の結果は右側の呼び出しで上書きされる。

これは第 03 回から使っている二項演算の退避パターンで解決する。

```

左辺を codegen する → a0 に fib(n-1) の結果
左辺の結果 a0 をスタックに退避する
右辺を codegen する → a0 に fib(n-2) の結果
退避していた左辺を a1 に戻す
add a0, a1, a0          → a0 = fib(n-1) + fib(n-2)

```

`ND_CALL` も普通の式と同じように結果を `a0` に残すため、既存の二項演算パターンがそのまま再帰呼び出しにも効く。

なお、右側の `fib(n - 2)` を呼ぶ時点では左辺の結果を 1 個積んだままである。前節のコメント調整が要るのは、まさにこの状態で `call` を出すときである。

9 関数プロトタイプ

相互再帰では、関数定義の前に関数宣言が現れることがある。

```
int is_even(int n);  
int is_odd(int n);
```

Parser はこれを `ND_FUNCPROTO` として返す。

```
(funcproto "is_even" :type int  
  (params (param "n" :type int)))
```

第 08 回では、関数プロトタイプに対してアセンブリを出す必要はない。`main()` の最後のループでは、`ND_FUNCDEF` だけを `gen_func()` すればよい。

10 gen_func の変更点

`gen_func(node)` に、引数関連の処理を追加する。新しいスキファールドでは、`gen_func` の流れは `_reset_func_state` → `_emit_func_prologue` → `_emit_func_body` → `_emit_func_epilogue` のフックメソッドで構成されている。

1. `_reset_func_state()` で `self._locals` と `self._stack_offset` を初期化する
2. `return` 用の共通ラベルを作る
3. `funcdef.params` を `self.alloc_local()` する ← 第 08 回で追加
4. `self.collect_decls(funcdef.body)` でローカル変数を集める
5. `frame_size = align_to(self._stack_offset, 16)` を計算する
6. プロローグを出す
7. `a0, a1, ...` を `params` のスロットへ `sd` する ← 第 08 回で追加
8. 関数本体の各文を `self.gen_stmt(node)` で出力する
9. `return` 用ラベルを出力する
10. エピローグを出力する

11 発展的な話題

この回では、関数呼び出しの基本だけを扱う。実際の C コンパイラや本格的な呼び出し規約では、さらに次のような話題がある。

話題	内容	この回での扱い
9 個以上の引数	<code>a0～a7</code> に入りきらない引数はスタックで渡す	扱わない
可変長引数	<code>printf</code> のように引数個数が変わる関数	扱わない
型チェック	宣言と呼び出しの引数個数・型が一致するか確認する	扱わない
外部関数呼び出し	libc の <code>printf</code> や <code>malloc</code> を呼ぶ	後の回で扱う
caller-saved / callee-saved	関数呼び出し前後でどのレジスタを誰が保存するか	この回では <code>a0～a7</code> , <code>ra</code> , <code>s0</code> の基本だけ扱う
末尾呼び出し最適化	<code>return f(x);</code> をジャンプに変える最適化	扱わない

これらは重要な話題だが、第 08 回ではまず、8 個以下の整数引数を持つユーザー定義関数を呼び出せることを目標にする。

12 編集するファイル

- `mycc.py`

`importlib` でコマ 07 の `Codegen07` を継承した `Codegen08` に、以下の機能を追加する (スケルトンにあらかじめ書かれている)。

主な追加・変更点は次の通り。

ハンドラメソッド	変更内容
<code>codegen_Call(node)</code>	関数呼び出しのコード生成 (新規)
<code>codegen(node)</code> の <code>match</code> 節に <code>'Call'</code>	ディスパッチャに追加する (スケルトンに書かれている)
<code>gen_func(node)</code>	パラメータ登録・引数退避を追加する。フックメソッド <code>_reset_func_state</code> , <code>_emit_func_prologue</code> , <code>_emit_func_body</code> , <code>_emit_func_epilogue</code> に処理を記述する

13 tests/

ファイル	内容	期待値
<code>target.c</code>	フィボナッチ再帰 <code>fib(10)</code>	55
<code>fact.c</code>	階乗再帰 <code>fact(5)</code>	120
<code>add_mul.c</code>	複合関数 (<code>add + mul</code>)	42
<code>sum_rec.c</code>	再帰総和 <code>sum(10)</code>	55

`call_add.c` | 単純な関数呼び出し | 対応する `.ans` を参照 |

`call_mul.c` | 乗算関数の呼び出し | 対応する `.ans` を参照 |

`three_args.c` | 3 引数関数の呼び出し | 対応する `.ans` を参照 |

`fib_rec.c` | 再帰フィボナッチ | 対応する `.ans` を参照 |

`fact_rec.c` | 再帰階乗 | 対応する `.ans` を参照 |

`mutual_rec.c` | 相互再帰 | 対応する `.ans` を参照 |

14 テスト

```
python3 scaffold/test_runner.py sessions/08_functions_recursion
```

`tests/target.c` がコンパイルでき、終了コード 55 になれば基本形は成功。

個別に動かす場合は、次のようにする。

```
python3 sessions/08_functions_recursion/mycc.py sessions/08_functions_recursion/te
| riscv64-linux-gnu-gcc -x assembler -static - -o out

qemu-riscv64 ./out
echo $?
```

15 注意

この回では、引数は最大 8 個まで扱う。9 個以上をレジスタ渡しできない場合のスタック渡しは扱わない。

`ND_CALL` の引数は 1 個もない場合もある。引数なしの関数呼び出しでは、引数を `push` せず、直接 `call` を出す。

`main` 関数も `ND_FUNCDEF` の 1 つである。`main` にはパラメータがないため、`gen_func()` の引数処理部分は特に影響しない。

`main` 以外の関数の `return` も、第 05 回と同じく共通エピローグラベルへジャンプする。各関数が独立した `_ret_label` を持つことに注意する。